

IICPC Platform — Architecture Blueprint

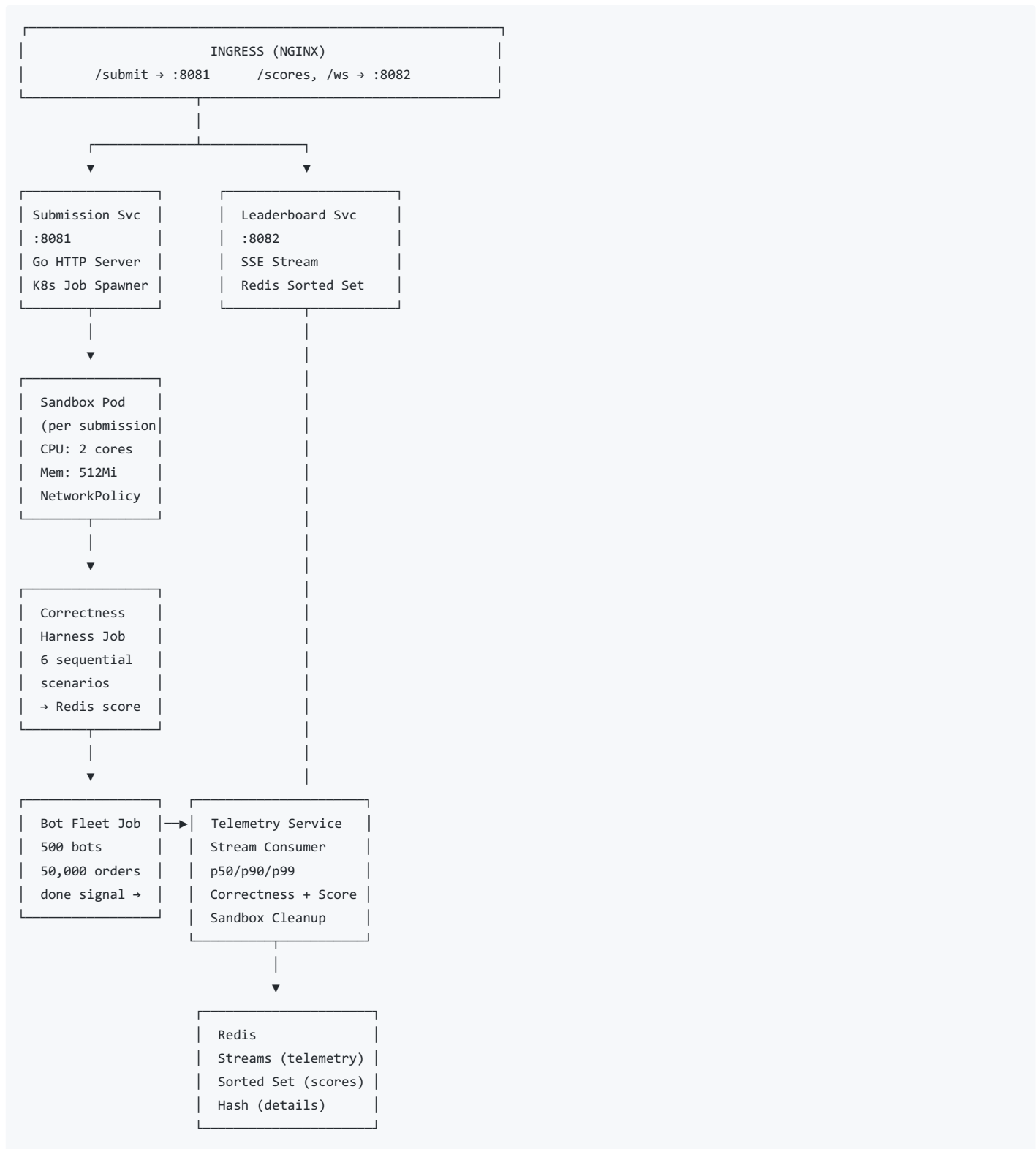
1. System Overview

The IICPC Distributed Benchmarking Platform is a cloud-native, microservices-based system designed to evaluate contestant-submitted orderbook implementations under simulated peak market conditions. The platform automates the complete evaluation pipeline — from code upload to real-time scoring — with strict isolation guarantees and objective performance measurement.

2. Design Goals

Goal	Requirement
Isolation	Each submission runs in its own K8s Job with CPU/memory limits and NetworkPolicy
Fairness	Identical resource allocation per contestant, no retries, TTL-enforced cleanup
Accuracy	Telemetry decoupled from bot fleet via Redis Streams — measurements not polluted by ingestion overhead
Scalability	Bot fleet scales 2→10 pods via HPA, platform handles 50 concurrent sandbox Jobs
Observability	p50/p90/p99 latency, TPS, fill correctness streamed live to leaderboard

3. Architecture Diagram



4. Microservices

4.1 Submission Service (:8081)

Responsibility: Accept contestant zip uploads, validate, persist to PVC, dynamically create K8s Jobs for sandbox execution.

Key design decisions:

- Uses `K8s client-go` to dynamically generate Job manifests per submission
- Each Job runs in `default` namespace with submission-scoped labels
- `BackoffLimit: 0` ensures no retries – fair competition
- `TTLSecondsAfterFinished: 300` auto-cleans Jobs after 5 minutes

Endpoints:

```
POST /submit    → upload zip, returns submission_id
GET  /status    → check sandbox status by submission_id
```

4.2 Sandbox Pod

Responsibility: Runs the contestant's Docker image in an isolated K8s Pod. Exposes `POST /order`, `DELETE /order`, `GET /orderbook` on port 8080. Automatically cleaned up by telemetry service after scoring completes.

4.3 Correctness Harness

Responsibility: Runs 6 deterministic sequential scenarios against the sandbox before load testing begins. Publishes a correctness score (0.0–1.0) to Redis.

Why sequential and not concurrent: Price-time priority correctness cannot be verified under concurrent load — the outcome depends on arrival order which is non-deterministic across the network. Sequential isolation is the only way to define a ground truth expected outcome.

Scenarios: basic price cross, no cross, cancel prevents fill, partial fill, market order, orderbook structure validation.

4.4 Bot Fleet

Responsibility: Simulate diverse market participants sending high-velocity concurrent orders to contestant's exchange.

Bot types:

Bot Type	Behavior	Distribution
MarketOrderBot	Sends market orders (immediate fill)	33%
LimitOrderBot	Sends limit orders at random prices 95-105	33%
CancelBot	Places then immediately cancels orders	33%

Concurrency model:

- 500 goroutines per pod, each bot is an independent goroutine
- Go's M:N scheduler maps goroutines across all CPU cores
- No shared state between bots — lock-free design
- Measured throughput: **12,158 orders/sec** on single laptop node

Why Go over Python: Python's GIL allows only one thread to execute bytecode at a time. Even with asyncio, all coroutines share one CPU core effectively. Go goroutines are multiplexed across all CPU cores by the runtime scheduler, achieving 10x higher throughput for concurrent HTTP workloads.

Scaling: K8s HPA scales bot-fleet pods 2→10 based on CPU utilization, enabling up to 1000 concurrent bots per test run.

4.5 Telemetry Service

Responsibility: Consume latency measurements from Redis Streams, calculate p50/p90/p99 percentiles and real TPS (using actual elapsed time), read correctness score from Redis, compute composite score, push to leaderboard. Deletes sandbox pod and service after scoring via K8s API.

Why Redis Streams over direct write:

If bots wrote directly to telemetry service:

```
100 bots → telemetry HTTP endpoint simultaneously
→ telemetry becomes bottleneck
→ bots wait for telemetry ACK
→ latency measurements include telemetry overhead
→ p99 numbers are wrong
```

With Redis Streams:

```
100 bots → Redis XAdd (microseconds, non-blocking)
→ bots measure only exchange latency
→ telemetry reads at own pace
→ measurements accurate
```

Percentile calculation:

```
Collect N latency samples → sort → index at N×percentile/100
p50 = sorted[N×0.50]    (median – typical experience)
p90 = sorted[N×0.90]    (90% of orders faster than this)
p99 = sorted[N×0.99]    (worst-case – stress indicator)
```

4.6 Leaderboard Service (:8082)

Responsibility: Serve current scores via REST and stream live updates via Server-Sent Events (SSE).

Data store: Redis Sorted Set (`ZADD leaderboard <score> <json>`)

- $O(\log N)$ insert
- $O(\log N + K)$ range query for top-K
- Natural ranking by score

Why SSE over WebSocket: SSE is unidirectional (server → client), simpler to implement, no handshake overhead, natively supported by browsers without libraries. WebSocket is bidirectional – unnecessary complexity for a read-only leaderboard feed.

Live update flow:

```
Telemetry writes score → Redis Sorted Set
Leaderboard SSE ticker (every 2s) → reads top 50 → pushes to browser
Browser EventSource receives → React state update → re-render
```

5. Data Stores

5.1 Redis

Usage 1 – Streams (telemetry pipeline):

```
Stream key: telemetry:orders
Producer:   bot-fleet (XAdd per order)
Consumer:   telemetry-service (XReadGroup, consumer group)
```

Consumer groups ensure exactly-once processing even if telemetry restarts.

Usage 2 – Sorted Set (leaderboard):

```
Key:      leaderboard
Score:    composite score (float64)
Member:   team_name (string) – one entry per team, best score kept
Details:  leaderboard:details hash – full JSON per team
```

Deployed as: K8s StatefulSet with PVC for persistence across restarts.

6. Inter-Service Communication

From	To	Protocol	Justification
Browser	Submission Svc	HTTP REST	Standard file upload
Browser	Leaderboard Svc	SSE	Live updates, unidirectional
Submission Svc	K8s API	HTTPS (client-go)	Dynamic Job creation
Bot Fleet	Sandbox	HTTP REST	Matches contestant API contract
Bot Fleet	Redis	Redis protocol	XAdd – non-blocking publish
Telemetry	Redis	Redis protocol	XReadGroup – consumer group

Leaderboard	Redis	Redis protocol	ZPerRange — sorted set query
Front	Telemetry	Protocol	Justification

Note on gRPC: gRPC was evaluated for bot→telemetry communication. Redis Streams was selected instead because it provides the same producer-consumer semantics with built-in persistence, consumer groups, and replay capability — without requiring proto compilation or gRPC server management in the hot path.

7. Kubernetes Architecture

7.1 Resource topology

```

Namespace: iicpc
├─ StatefulSet: redis (1 replica)
├─ Deployment: submission-service (1 replica)
├─ Deployment: telemetry-service (1 replica)
├─ Deployment: leaderboard-service (1 replica)
├─ Jobs:      sandbox-<submission-id> (1 per submission, TTL 300s)
├─ Jobs:      bot-fleet-job (triggered per test run)
├─ Jobs:      correctness-<submission-id> (1 per submission, TTL 300s)
├─ HPA:       bot-fleet (2→10 replicas)
├─ ResourceQuota: max 50 concurrent Jobs, 20 CPU, 10Gi memory
├─ NetworkPolicy: sandbox pods — ingress from bot-fleet only
└─ Ingress:    NGINX — routes /submit, /scores, /ws

```

7.2 Sandbox isolation layers

```

Layer 1 — Namespace isolation
  Each sandbox Job labeled with submission-id

Layer 2 — Resource limits
  CPU: 2 cores hard limit (no burst)
  Memory: 512Mi hard limit (OOMKilled if exceeded)

Layer 3 — Security context
  runAsNonRoot: true
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false

Layer 4 — Network isolation
  NetworkPolicy: only bot-fleet → sandbox:8080 allowed
  All sandbox egress blocked

```

8. Scoring Formula

```

Score = (TPS_normalized × 0.4) + (Latency_score × 0.4) + (Correctness × 0.2)

Where:
  TPS_normalized   = orders_per_second
  Latency_score    = (1 / p99_ms) × 1000
  Correctness      = fill_accuracy% (price-time priority validation)

```

Rationale:

- TPS (40%) — measures raw throughput capacity
- Latency (40%) — p99 specifically, not p50, because consistency under stress matters more than median performance in HFT systems
- Correctness (20%) — a fast but incorrect orderbook is disqualified from real trading; score penalizes fill errors

9. Performance Benchmarks

Measured on HP ProBook G10, AMD Ryzen, 14GB RAM, single minikube node:

Metric	Value
Bot fleet throughput	12,158 orders/sec
p50 latency (mock exchange)	9ms
p90 latency (mock exchange)	17ms
p99 latency (mock exchange)	27ms
Goroutines per bot-fleet pod	100
K8s Job spawn time	~3 seconds
Redis XAdd throughput	>100,000 ops/sec

10. Security Considerations

Threat	Mitigation
Malicious code execution	readOnlyRootFilesystem, runAsNonRoot, no shell access
Resource exhaustion (CPU bomb)	Hard CPU/memory limits via ResourceQuota
Network attacks from sandbox	NetworkPolicy blocks all sandbox egress
Zip bomb / oversized upload	50MB upload limit enforced at submission service
Container escape	Non-root execution, no privileged containers
Data tampering	Each submission isolated in separate Job namespace

CORS

Current configuration allows all origins (`*`) suitable for local development and hackathon demo. For production deployment with a domain, update the CORS origin in 3 files:

- backend/submission-service/main.go
- backend/leaderboard-service/main.go
- mock-exchange/main.go

Change: `w.Header().Set("Access-Control-Allow-Origin", "*")` To: `w.Header().Set("Access-Control-Allow-Origin", "https://your-domain.com")`

Sandbox Isolation

Contestant code runs with zero network egress via K8s NetworkPolicy. Malicious outbound calls from submitted code are blocked at the infrastructure level regardless of what the code attempts.